Unveiling LLM Hallucination in Recommending Third-party Packages

ANONYMOUS AUTHOR(S)

In this paper, ...

1 Introduction

1. LLM widely used for developers
   2. LLM hallucination
   3. Hallucinated packages can cause supply chain attack.
   4. How hallucinated package is not yet fully explored
   5. The purpose of this study is to.
   6. Specifically, we design xxx.
   7. The results reveal that
   8. Our contribution are as follows.

2 Background

2.1 Hallucination

2.2 Package Squatting

3 Empirical Study Design

3.1 Research Questions

3.2 Dataset Setup

For the investigation of RQ1 and RQ2, we constructed a functional question dataset designed to trigger package recommendation behaviors in large language models. Furthermore, we developed a semantically rewritten version of this dataset to mitigate the potential impact of data leakage.

3.2.1 Initial Dataset Collection. To conduct an in-depth analysis of the hallucination phenomenon that may occur when large language models recommend Python packages, this study first constructed a functional question dataset covering typical development task scenarios. The dataset was sourced from Stack Overflow, where we systematically identified questions related to major Python application domains that have the potential to trigger package recommendation behaviors, thereby establishing a reliable data foundation for the subsequent empirical study.

First, we adopted the co-occurrence-based tag filtering method proposed by Huang et al. [1] and utilized the publicly available Stack Overflow dataset provided by Google BigQuery [2] to extract all tags that co-occurred with the "python" tag more than 1,000 times, resulting in a total of 212 tags.On this basis, drawing on the topic classification strategy used by Du et al. [3] in constructing code generation benchmark tasks, as well as the categorization of common Python development topics summarized by Tahmooresi et al. [4], we manually reviewed these 212 high-frequency co-occurring tags. Specifically, we assessed whether each tag belonged to one of the twelve mainstream Python development domains, whether it was likely to involve the use or recommendation of software packages, and whether it could potentially give rise to functional questions of the form "how to implement a specific functionality."Through this filtering process, we ultimately retained 114 tags. Their detailed categorization is presented in Table 1.

After completing the tag filtering process, we extracted from the public dataset all questions that contained both the "python" tag and at least one of the filtered tags, resulting in a total of 287,944 questions. Subsequently, we randomly sampled 200 questions for manual analysis. The results indicated that typical functional questions related to software package recommendation behaviors often contain functional expression keywords such as "how to" or "how can I."Based on this observation, we filtered the entire set of questions using these keywords, obtaining 49,967 candidate questions with potential functional requirements.

To further identify genuinely functional questions from the 49,967 candidate questions, we first conducted an in-depth analysis of a randomly selected subset of 200 questions and summarized the typical characteristics of functional questions. Based on these findings, we designed targeted classification prompts and invoked the large language model DeepSeek R1 to evaluate each candidate question individually. Through this process, we obtained a refined dataset consisting of 17,696 functional questions. The detailed design of the prompts is provided in Appendix A.

After completing the preliminary screening of functional questions, we further removed those without an accepted answer to ensure that each retained question was associated with a community-validated solution. Subsequently, we extracted the accepted answer for each question from the public dataset and matched it with the corresponding question, thereby constructing a question–answer pair dataset for subsequent analysis.

To determine whether each accepted answer involved specific third-party Python libraries, we designed corresponding prompts (the details of which are provided in Appendix B) and then employed the large language model DeepSeek R1 to analyze all accepted answers, automatically extracting the names of third-party libraries mentioned in the responses (i.e., software packages that can be installed from PyPI via pip).For question–answer pairs where the model returned an empty result—indicating that the answer did not involve any third-party libraries—we excluded them from subsequent processing. After this filtering step, we ultimately obtained 4,382 functional question–answer pairs containing information on third-party library usage, which constitute the final functional question dataset used in this study.

3.2.2 Semantic Rewriting Dataset Construction. Since the above question–answer data were all sourced from the publicly available Stack Overflow community, some of the content may have been encountered by large language models during their training process. Therefore, directly conducting evaluations based on the original questions may underestimate the actual occurrence rate of hallucinated packages.To obtain more reliable experimental results, we performed semantic rewriting on all questions to mitigate the potential impact of data leakage.

Specifically, we invoked the large language model DeepSeek R1 to generate five semantically equivalent rewritten versions for each question. To achieve a balance between semantic preservation and variation in expression—avoiding rewrites that were either overly similar to the original question or that deviated excessively in meaning—we further introduced a filtering strategy based on sentence embedding similarity.For each original question and its five corresponding rewrites, we employed the Sentence-BERT model all-mpnet-base-v2 to generate sentence embeddings and computed the cosine similarity between the embedding of the original question and that of each rewritten version, resulting in five similarity scores. We then ranked the five rewritten versions in descending order of similarity and selected the version whose similarity score was at the median position as the final semantic rewrite for that question.Ultimately, we constructed a semantically rewritten question dataset that

corresponds one-to-one with the original functional question dataset, which was used for subsequent evaluation of hallucinated packages under controlled conditions to mitigate potential data leakage effects.

In summary, this study constructed a functional question–answer dataset covering multiple typical Python development scenarios based on publicly available Stack Overflow data, along with a corresponding semantically rewritten question dataset.

3.3 RQ Setup

3.3.1 RQ1 Setup. To evaluate the frequency with which large language models generate hallucinated packages when recommending Python libraries, we designed a unified and comprehensive experimental procedure. We selected several mainstream models, including ChatGPT-4, DeepSeek-R1, Qwen3, and Claude-Opus-4. The titles of the questions from the functional question dataset constructed in the previous stage were used as input prompts, and each model was asked to recommend one or more third-party Python libraries that could be used to address the given question.To distinguish cases in which a model determines that no third-party library is required, we specified in the prompt that if the model judges a question does not require any third-party libraries, it should return a special token, `no_package`.

During the experiment, we first designed a unified prompt for all models, the full content of which is provided in Appendix C.When querying the models, we observed that some of them occasionally returned package names containing characters not permitted by PyPI (e.g., parentheses, square brackets, equal signs, or spaces). To ensure the validity and consistency of the experimental results, we implemented an automated script to examine the model outputs and automatically reissue the query whenever illegal characters were detected, repeating this process until the returned package names contained no disallowed characters.

In addition, for some package names generated by the models that contained dot characters (e.g., `scipy.sparse`), we found that such submodules cannot be directly installed as top-level packages. To address this issue, we designed a more restrictive prompt (see Appendix D), explicitly requiring the models to return only top-level package names—for example, replacing `scipy.sparse` with `scipy`. Whenever a returned result contained a dot, we reissued the query using this stricter prompt.However, we further observed that even after multiple rounds of querying, a small number of models occasionally still returned package names containing dots (e.g., *RPi.GPIO*). In such cases, we chose to retain these results and handle them uniformly in the subsequent stages of the workflow.

After completing the queries to all models, we performed a unified post-processing procedure on the returned package names. First, we removed all records whose returned value was `no_package`, retaining only those questions for which the models explicitly provided recommended package names.Subsequently, we queried each recommended package individually using the official PyPI JSON API (https://pypi.org/pypi/\{pkg_name\}/json) to verify whether it actually exists. When the request returned a status code of 200, we regarded the package as a real package.During this process, we observed that the previously retained package names containing dot characters also returned a status code of 200 from the API, indicating that these dotted package names indeed exist on PyPI. Therefore, we treated them as real packages in our analysis as well.

Finally, we conducted an additional screening of the package names that failed the PyPI verification and were initially identified as hallucinated packages. Although the prompts explicitly required the models to recommend only third-party libraries, some models occasionally returned Python standard libraries (e.g., *os*, *sys*) for a small number of questions. Since these standard libraries are not hosted on PyPI, they would be misclassified

as hallucinated packages in the previous verification step.To eliminate such cases, we utilized the `importlib.util.find_spec()` interface to determine whether a given name belongs to the Python standard library and removed all identified standard library entries accordingly. After this filtering step, we ultimately obtained a result set containing only hallucinated packages.

3.3.2 RQ2 Setup. To further investigate whether hallucinated packages generated by large language models pose a risk of being downloaded by real developers, we selected all 91 hallucinated package names recommended by ChatGPT-4 and attempted to register and upload them to the PyPI platform.For each package, we created standardized `setup.py`, `__init__.py`, and `README.md` files.The file contents explicitly stated that the package was intended solely for academic research purposes, contained no functional code, and performed no malicious operations. The README file provided detailed information regarding the research background, research participants, contact information, and research objectives to ensure that the experimental process complied with academic ethical standards. All packages were uploaded manually by the authors.

We began uploading these hallucinated packages on May 18, 2025, and completed the entire upload process on May 22, 2025. Throughout this period, a total of nine PyPI accounts were used to register and upload the packages in batches.Among the 91 packages, 55 were successfully registered, while the remaining 36 failed due to various reasons. The causes of failure included certain package names being protected by PyPI, names being considered too similar to existing projects, and account restrictions imposed by the platform due to bulk upload activities being flagged as suspicious behavior.

On June 10, 2025, we received an email from the PyPI administrators requesting the immediate removal of all packages related to the experiment. Upon receiving the notification, we completed the removal of all packages between June 12 and June 13.

After uploading the hallucinated packages to PyPI, we continuously monitored their subsequent download activities to assess their real-world impact. We collected download statistics for each package using two third-party data analytics platforms, pepy.tech and pypistats.org, which provide daily download counts as well as cumulative downloads over different time ranges. The corresponding results and analyses will be presented in detail in the Results section.

3.3.3 RQ3 Setup.

4 Current Status and Characteristics (RQ1)

4.1 RQ1.1 How Frequently Do Different Large Language Models Generate Hallucinated Packages in Real Development Scenarios, and What Are the Distribution Characteristics of the Generated Packages? Are There Significant Differences Across Models?

To compare the frequency and potential risk of hallucinated package generation by different large language models in real development scenarios, we counted the number of hallucinated packages generated by four models on the functional question dataset, as well as their corresponding proportions, as shown in Fig. 1. It should be noted that this study focuses on the potential misleading risk that models may pose to developers when recommending software packages in practice. Therefore, when calculating the hallucination ratio, we used the number of questions for which the model explicitly recommended at least one package name as the denominator, and excluded questions where the model returned *no_package*.

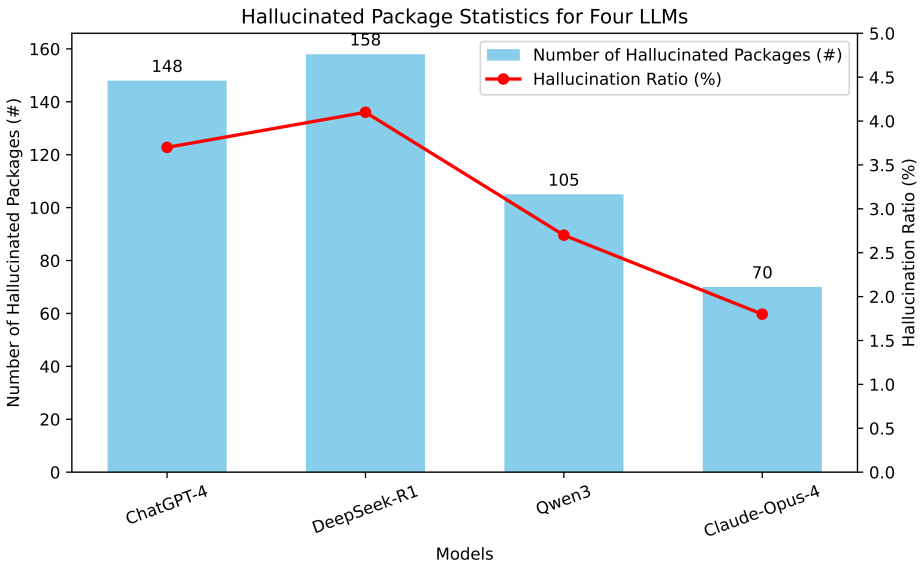

Fig. 1. Number and Ratio of Hallucinated Packages Generated by Different Large Language Models

This is because in such cases the model did not perform any package recommendation and thus would not directly affect developers.

From the statistical results, it can be observed that all evaluated large language models generate hallucinated packages for a certain proportion of questions, indicating that this phenomenon is common across different models. On this basis, different models exhibit relatively significant differences in both the number and the proportion of hallucinated packages generated. Among them, DeepSeek-R1 generates the largest number of hallucinated packages and also has the highest hallucination ratio, reaching 4.02%; ChatGPT-4 ranks second, while Qwen3 and Claude-Opus-4 generate relatively fewer hallucinated packages. However, even for Claude-Opus-4, which has the lowest hallucination ratio among the evaluated models, the ratio still reaches 1.72%, indicating that hallucinated packages are not caused by a few individual models or extreme cases, but rather represent a potential risk that commonly exists when large language models perform software package recommendation tasks.

After analyzing the frequency of hallucinated packages generated by different large language models, we further examined the consistency relationships among the hallucinated packages generated by these models. Specifically, we compared the sets of hallucinated packages generated by the four models and conducted an intersection–union analysis to examine whether there exist hallucinated packages that are simultaneously generated by multiple models, as shown in Fig. 2.

The results show that the vast majority of hallucinated packages are generated by only a single model, and the overall overlap among different models is relatively low. Only a very small number of hallucinated packages are recommended simultaneously by two or more models. This result indicates that the generation of hallucinated packages does not mainly originate from a few “highly misleading” fixed package names, but is more likely related to the randomness during the generation process, inference biases, or differences in contextual

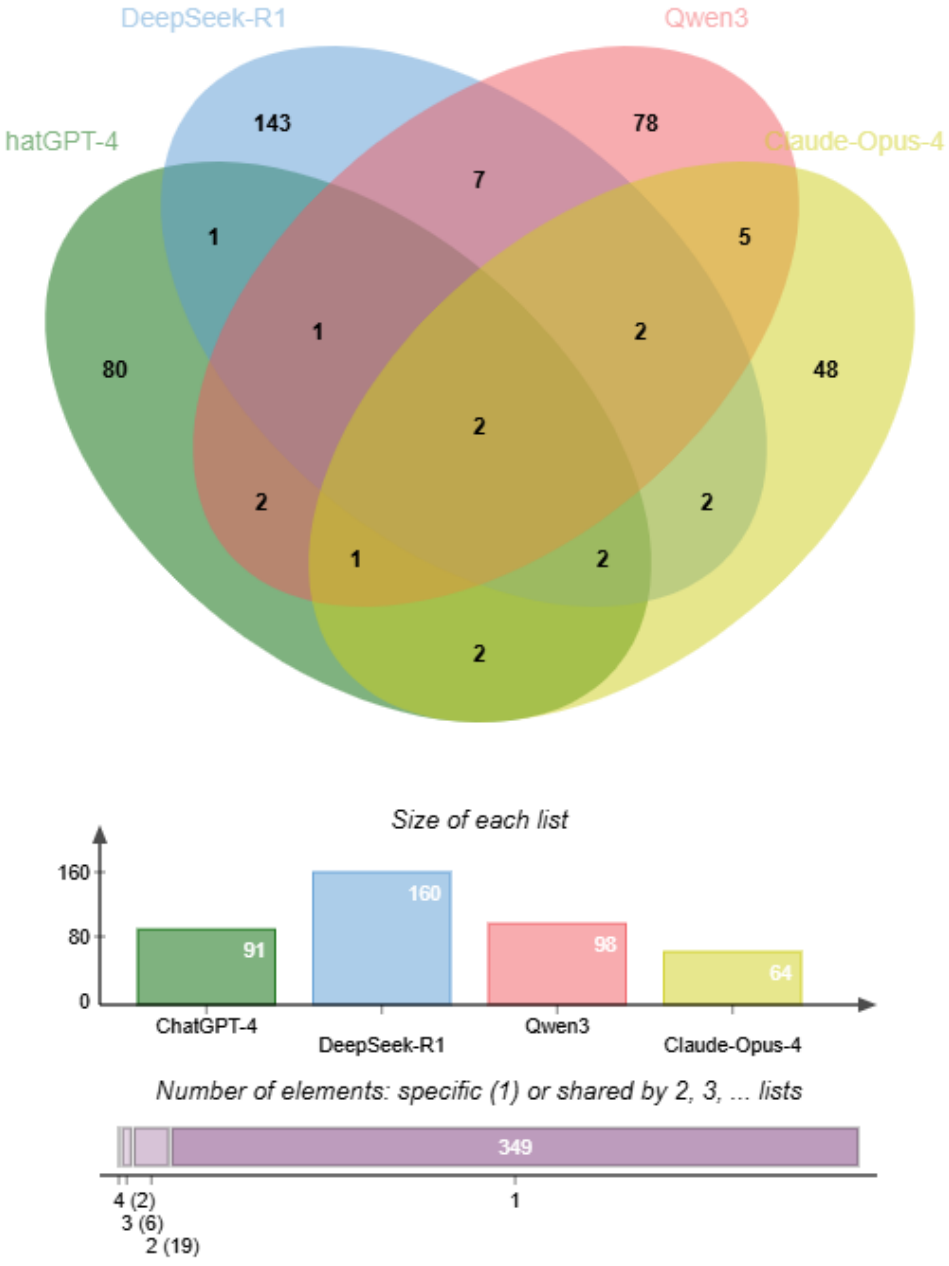

Fig. 2. Intersection and Union Distribution of Hallucinated Packages Generated by Different Large Language Models

understanding across models. Therefore, the generation behavior of hallucinated packages exhibits strong model-specific characteristics across different models.

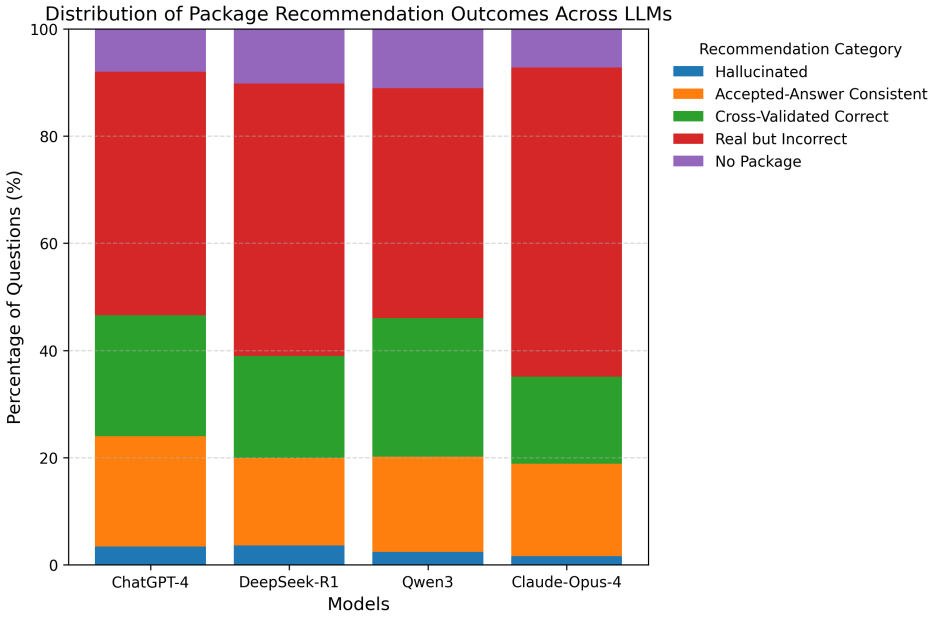

Fig. 3. Category Distribution of Package Recommendation Outcomes Across Different Large Language Models

Existing studies that analyze the problem of hallucinated packages recommended by large language models often mainly focus on the proportion of hallucinated packages generated. However, examining the issue solely from the single perspective of hallucinated packages is insufficient to comprehensively characterize the overall behavioral patterns of models in software package recommendation tasks. In addition to hallucinated packages, whether the packages recommended by a model are consistent with the real solutions, or at least correspond to reasonable and usable third-party packages, is also an important aspect for understanding its recommendation behavior, and helps us analyze the underlying mechanisms of hallucinated package generation from a more comprehensive perspective. Based on this consideration, we further analyze the distribution of recommendation outcomes of different large language models in package recommendation tasks from an overall perspective.

Specifically, for each question, we denote the set of third-party software packages used in the accepted answer as A, which we regard as the set of packages adopted by the correct solution for that question in real development scenarios. We denote the set of high-confidence correct packages obtained through cross-validation across multiple models as C, where each package must satisfy two conditions: it is consistently recommended by at least three different large language models and is verified by PyPI to be a real existing package rather than a hallucinated one. We denote the set of packages actually recommended by the model as P, among which the subset identified as hallucinated packages is denoted as H. Based on these definitions, for each question we categorize the model's recommendation outcome into the following five cases:

(1) **Hallucinated**: when $H \neq$, the model generates at least one hallucinated package;

(2) **Accepted-Answer Consistent**: when $P \subseteq A$, the packages recommended by the model are completely consistent with the packages used in the accepted answer;

(3) **Cross-Validated Correct**: when $P \subseteq A \cup C \ \wedge \ P \nsubseteq A$, the packages recommended by the model do not all appear in the accepted answer, but are supported by cross-validation across multiple models;

(4) **Real but Incorrect**: when all packages in P are real packages, but at least one package does not belong to $A \cup C$;

(5) **No Package**: the model returns *no_package*, meaning that the model does not provide any third-party package recommendation.

Based on the above classification rules, we calculated the proportional distribution of the five categories of package recommendation outcomes for the four large language models, as shown in Fig. 3. From the overall results, *Real but Incorrect* occupies the highest proportion across all models, and its proportion is overall higher than the combined proportion of the two correct recommendation categories, *Accepted-Answer Consistent* and *Cross-Validated Correct*. This indicates that, compared with completely fabricated package names that do not exist, a more common error form in package recommendation tasks is that models recommend packages that actually exist but are not suitable for the current problem. This type of error occupies the main proportion in the overall recommendation behavior.Meanwhile, all four models return *no_package* for a certain proportion of questions. This phenomenon indicates that when the model cannot determine an appropriate third-party package, it does not always forcibly provide a recommendation, but instead demonstrates a certain degree of conservativeness in its responses, which can to some extent avoid arbitrary guessing. This reflects a trade-off between "recommendation activeness" and "answer cautiousness" in software package recommendation tasks.In addition, although the proportion of *Hallucinated* in the overall distribution is relatively small, it still appears across all models. Moreover, its potential harm differs from other error types, as it may cause additional debugging costs for developers and even introduce risks to the software supply chain. Therefore, it still has non-negligible research significance.

Beyond the overall distribution characteristics, the proportions of the five recommendation outcome categories also exhibit noticeable differences across different models, reflecting different tendencies in package recommendation strategies among models. On the one hand, the proportions of *No Package* and *Real but Incorrect* differ across models: some models choose to return *no_package* for a larger number of questions, demonstrating a more conservative recommendation tendency; whereas other models tend to provide specific package recommendations, which also to some extent increases the proportion of recommending real but unsuitable packages.On the other hand, differences also exist in the overall proportions between recommending correct packages and incorrect packages across models. Some models have higher proportions in the two categories *Accepted-Answer Consistent* and *Cross-Validated Correct* compared with other models, indicating that they are overall more likely to recommend correct packages. In contrast, other models relatively more frequently recommend packages in the *Real but Incorrect* and *Hallucinated* categories, indicating that they are more likely to produce inappropriate recommendations in package recommendation tasks.These differences indicate that under the same question set and unified prompting conditions, the overall reliability levels of different large language models in software package recommendation tasks are not consistent.

Based on the above analyses, we can draw the following conclusion: different large language models inevitably generate a certain proportion of hallucinated packages in software package recommendation tasks under real development scenarios, indicating that this phenomenon has a certain degree of universality. Meanwhile, further intersection–union analysis shows that hallucinated packages are mostly model-specific rather than concentrated in a few fixed

package name patterns. In addition, from the perspective of the overall recommendation outcomes, large language models are more likely to recommend packages that do not match the requirements of the current problem in software package recommendation tasks (including both hallucinated packages and real packages that are not suitable for the current problem), which reflects a certain degree of reliability deficiency in their recommendation decision process.

4.2 RQ1.2 What Patterns Can Be Observed in the Naming Structures, Semantic Sources, and Domain Distributions of Hallucinated Packages Generated by Large Language Models?

4.3 RQ1.3 Do Questions That Lead to Hallucinated Packages Exhibit Specific Semantic Characteristics? Which Types of Questions Are More Likely to Trigger Hallucinated Packages?

In the previous sections, we mainly analyzed the phenomenon of hallucinated packages, including the frequency of hallucinated package generation, the differences across different models, and the overall distribution characteristics of package recommendation outcomes. We further examined the naming structures, semantic sources, and domain distribution patterns of hallucinated packages. However, examining the issue solely from the perspectives of model outputs and the hallucinated packages themselves is still insufficient to fully explain the underlying causes of hallucinated package generation. Whether the questions that lead to hallucinated packages themselves possess potential characteristics is also worth investigating. Therefore, in this subsection we analyze whether the questions that produce hallucinated packages exhibit certain specific characteristics that make large language models more likely to generate hallucinations on these questions.

To answer this question, we first counted the number of times that each question triggered hallucinated package recommendations across the four large language models (ChatGPT-4, DeepSeek-R1, Qwen3, and Claude-Opus-4), and used this count as an indicator to measure the hallucination degree of each question. This indicator reflects whether different models exhibit consistent tendencies to generate hallucinated packages when facing the same question. Based on this statistical result, we categorized the questions into different groups according to the number of large language models that triggered hallucinated packages for each question, denoted as H.

In this dataset, the numbers of questions in the four groups with $H = 0$, $H = 1$, $H = 2$, and $H = 3$ are 3979, 333, 62, and 8, respectively. It should be noted that among all questions, there is no case in which hallucinated packages are recommended simultaneously by all four large language models. Based on the above grouping method, we conduct a systematic analysis of the potential differences in how different questions trigger hallucinated packages from three aspects: input complexity of the questions, global semantic structure, and local semantic neighborhood structure.

4.3.1 Input Complexity Analysis. Based on the above grouping, we first examine whether the input complexity of questions is related to the generation of hallucinated packages. Considering that large language models process inputs using tokens rather than character counts during actual inference, this study adopts the *tiktoken* tool provided by OpenAI to tokenize each question and calculate the corresponding number of tokens, which is used as the metric for measuring the input complexity of questions.

On this basis, we further analyzed the distribution of token counts across questions with different hallucination intensities. Table 1 summarizes the number of questions in each

Table 1. Statistics of Input Complexity and Local Semantic Features for Questions with Different Hallucination Intensities

Hallucination Count (H)	Question Count	Avg. Token Count	Median Token Count	Neighbor F
0	3979	12.278	12.000	
1	333	11.733	11.000	
2	62	11.984	11.500	
3	8	13.625	12.500	

hallucination intensity group, along with the corresponding average token counts and median token counts. From the overall results, the input complexity across different hallucination intensity groups does not exhibit an obvious monotonic pattern, and the differences among the groups are overall relatively limited.

To further examine whether the differences in input complexity among questions with different hallucination intensities are statistically significant, we employed the Kruskal–Wallis nonparametric test to compare the token count distributions across the four groups of questions. The test results show that the differences in token counts among the hallucination intensity groups do not reach statistical significance ($p = 0.052$).

This result indicates that, within the dataset used in this study, there is insufficient evidence to conclude that the input complexity of a question significantly affects the likelihood of large language models generating hallucinated packages. From this perspective, the occurrence of hallucinated packages cannot be simply attributed to increases in question length or the amount of input information. Instead, whether a question triggers hallucinated packages may be more closely related to its semantic expression patterns and deeper semantic structural characteristics. Therefore, in the subsequent analysis, we further investigate the questions from the perspective of semantic structure.

4.3.2 Global Semantic Structure Analysis. In the previous subsection, we found that the input complexity of questions cannot sufficiently explain the differences in the generation of hallucinated packages. Based on this finding, we further examine the distribution of questions with different hallucination intensities in the semantic space from the perspective of global semantic structure, in order to analyze whether the occurrence of hallucinated packages is related to the semantic topics or broader semantic regions to which the questions belong.

Specifically, we employ the Sentence-BERT model *all-mpnet-base-v2* to generate sentence embeddings for each question, thereby capturing the semantic characteristics of the questions in a high-dimensional semantic space. Subsequently, we apply the UMAP method to reduce the dimensionality of the high-dimensional semantic vectors and project them into a two-dimensional space for visualization. During the visualization process, questions are color-coded according to the number of large language models that generated hallucinated packages for each question (H), allowing us to observe the distribution differences of questions with different hallucination intensities in the global semantic space.

Figure 4 presents the projection of questions with different hallucination intensities in the UMAP semantic space. From the overall distribution, the questions form several continuous and dense regions in the semantic space. However, questions with different hallucination intensities do not form clearly separable clusters or boundaries within these semantic regions. Questions that trigger hallucinated packages ($H \geq 1$) are highly intermixed with those that

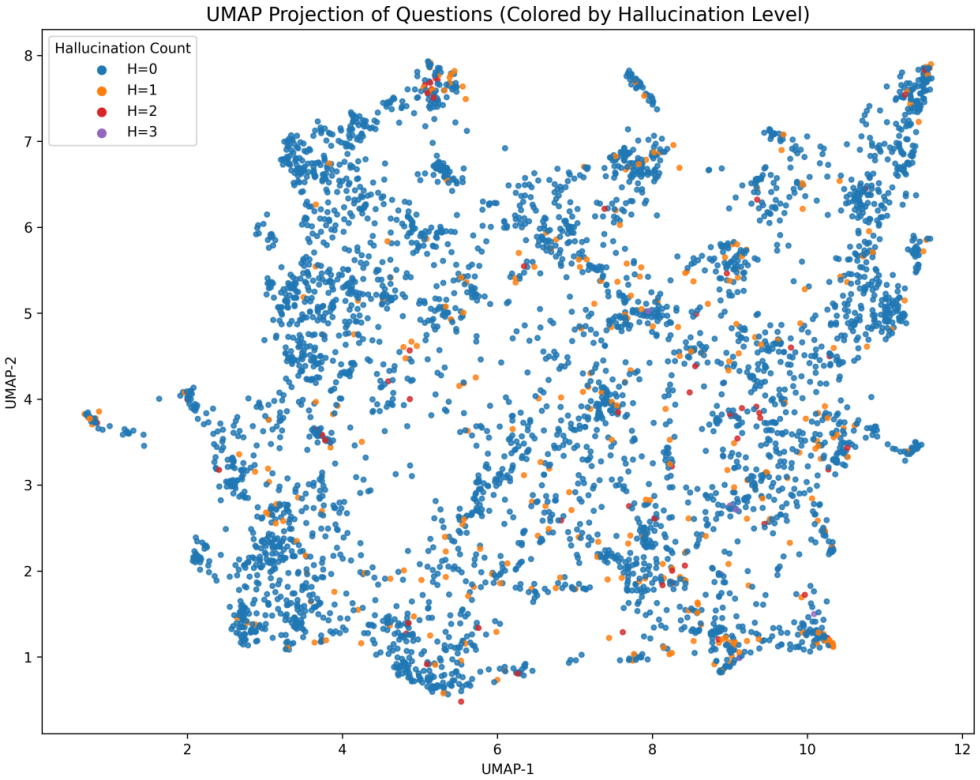

Fig. 4. UMAP Visualization of Questions in the Semantic Space Colored by Hallucination Intensity

do not trigger hallucinated packages ($H = 0$), and they do not appear to concentrate in any specific semantic region or topic cluster.

This result indicates that the occurrence of hallucinated packages is not concentrated within a small number of specific semantic topics or question types. In other words, from the perspective of global semantic distribution, questions that trigger hallucinated packages do not exhibit a clear "topic bias" or "domain concentration." Instead, the hallucinated package phenomenon is more likely to be a behavior that occurs broadly across different semantic regions, rather than being systematically triggered by certain specific semantic topics.

4.3.3 Local Semantic Neighborhood Structure Analysis. In the preceding analyses, we found that neither the input complexity of questions nor their positions in the global semantic space are sufficient to explain the differences in hallucinated package generation. Based on this observation, we further investigate the phenomenon from the perspective of local semantic structure, examining whether the generation of hallucinated packages exhibits potential correlations within neighborhoods of semantically highly similar questions.

Specifically, we use the Sentence-BERT model *all-mpnet-base-v2* to generate sentence embeddings for each question, in order to capture the semantic characteristics of questions in the high-dimensional semantic space. Subsequently, for each question in the dataset, we retrieve the 20 most semantically similar questions in the embedding space based on cosine similarity. The average number of large language models that generate hallucinated packages among these semantically similar questions is then used as the neighborhood hallucination

value for the given question. This metric characterizes the overall degree to which the hallucinated package phenomenon occurs within the set of questions that are highly similar to a given question in semantic space.

Based on this, we further group the questions according to the number of large language models that generate hallucinated packages for each question (H), and analyze the distribution of neighborhood hallucination values across different hallucination intensity groups. The statistical results are summarized in Table 1. From Table 1, it can be observed that questions that trigger hallucinated packages ($H \geq 1$) and those that do not trigger hallucinated packages ($H = 0$) exhibit relatively clear differences in their neighborhood hallucination values. Specifically, questions that do not trigger hallucinated packages ($H = 0$) have the lowest neighborhood hallucination values, whereas the $H = 2$ group shows the highest average level of neighborhood hallucination.

To further examine whether the differences in neighborhood hallucination values among questions with different hallucination intensities are statistically significant, we employ the Kruskal–Wallis nonparametric test to compare the four groups of questions. The test results indicate that the differences in the average neighborhood hallucination values among the groups are highly statistically significant ($p < 0.001$).

This result indicates that the generation of hallucinated packages is neither entirely random nor an occasional phenomenon independent of the semantic structure of questions. Although questions that trigger hallucinated packages do not exhibit clear topic-level clustering in the global semantic space, the hallucinated package phenomenon shows a significant degree of consistency within local neighborhoods of semantically similar questions. This suggests that hallucinated packages are more likely related to the expression patterns or implicit semantic details of questions at the local semantic level. It should be noted that we do not further investigate these specific semantic factors; rather, through empirical analysis we reveal the existence of such local correlations. A deeper investigation into the underlying semantic mechanisms behind this phenomenon remains a promising direction for future research.

5 Mitigation

5.1 Mitigation Methodology

5.1.1 *Enhanced Prompting.*

5.1.2 *Machine-learning-based Detection.*

5.1.3 *Context-Augmented-Prompt Validation.* Typo squatted package?

5.2 Evaluation Setup

- **Mitigation Effectiveness (RQ5):** How do mitigation techniques address the hallucinated package problem?

5.3 RQ5

5.4 Reservation of Squating/Hallucination Packages

6 Related Work

7 Conclusion

References

Table 2. AUROC (%) of different methods under varying member-to-non-member ratios.

Ratio	Method	Pythia 2.8B	GPT-Neo 2.7B	StableLM-Alpha 3B	GPT-J 6B
1:1	Loss	38.4	43.7	33.3	43.3
	ZLib	33.2	35.5	32.0	35.7
	Min-K	38.8	41.8	34.2	41.1
	DC-PDD	50.1	41.2	43.7	41.3
	tool	61.3	59.7	61.3	59.7
1:5	Loss	40.3	45.3	33.9	44.8
	ZLib	34.0	36.2	32.5	36.3
	Min-K	40.2	43.3	34.9	42.4
	DC-PDD	51.8	44.0	44.6	43.2
	tool	61.2	59.4	61.1	61.5
5:1	Loss	40.0	43.8	34.9	43.6
	ZLib	33.1	35.0	31.9	35.2
	Min-K	39.9	42.2	35.1	41.6
	DC-PDD	49.4	40.9	44.1	39.6
	tool	62.0	60.7	62.0	63.1